

Give `std::optional` Range Support

Document number: P3168R0

Date: 2024-02-28

Authors: Marco Foco <marco.foco@gmail.com (mailto:marco.foco@gmail.com)>, Darius Neațu <dariusn@adobe.com (mailto:dariusn@adobe.com)>, Barry Revzin <barry.revzin@gmail.com (mailto:barry.revzin@gmail.com)>, David Sankel <dsankel@adobe.com (mailto:dsankel@adobe.com)>

Audience: Library Evolution

Abstract

In *P1255R12: A view of 0 or 1 elements: `views::maybe`* (<https://wg21.link/P1255R12>), Steve Downey explores the benefits of an optional type that models the range concept and concludes that a new optional type with range support, `std::views::maybe`, should be introduced into the standard library. This paper explores the design alternative where, instead of introducing a new type, `std::optional` is made into a range. Observing that this improves usage examples and is dramatically more straightforward, we recommend this approach over that of *P1255R12*.

```
// A person's attributes (e.g., eye color). All attributes are optional.
class Person {
    /* ... */
public:
    optional<string> eye_color() const;
};

vector<Person> people = ...;
```

P1255R12

```
// Compute eye colors of 'people'.
vector<string> eye_colors = people
    | views::transform(&Person::eye_color)
    | views::transform(views::nullable)
    | views::join
    | ranges::to<set>()
    | ranges::to<vector>();
```

This Proposal

```
// Compute eye colors of 'people'.
vector<string> eye_colors = people
    | views::transform(&Person::eye_color)
    // no extra wrapping necessary
    | views::join
    | ranges::to<set>()
    | ranges::to<vector>();
```

Introduction

`std::optional<T>`, a class template that "may or may not store a value of type `T` in its storage space" was introduced by Fernando Cacciola and Andrezej Krzemiński's *A proposal to add a utility class to represent optional objects (N3793)* (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3793.html>). This feature was incorporated into C++17 and has since seen frequent use in data members, function parameters, and function return types.

Ranges were introduced into C++20 with Eric Niebler et al.'s *The One Ranges Proposal (P0896R4)* (<https://wg21.link/p0896r4>) as a set of standard library concepts and algorithms that simplify algorithm composition and usage.

As usage experience of both `std::optional` and ranges has grown, it became apparent that treating `std::optional` objects as ranges is quite useful, especially in algorithmic contexts. Steve Downey in *A view of 0 or 1 elements: `views::maybe` (P1255R12)* (<https://wg21.link/p1255r12>) showcases many examples where algorithm readability improves when `std::optional` objects are treated as ranges.

Downey proposed two facilities that integrate `std::optional` and ranges:

1. `views::nullable`. A range adapter producing a view over `std::optional` objects and, more generally, any dereferencable object with a `bool` conversion.
2. `views::maybe`. A data type with the same semantics as `std::optional` with the some key interface differences. We will discuss these differences a little later.

Downey suggests `std::maybe_view` be used instead of `std::optional` for return types and other situations where "the value will have operations applied if present, and ignored otherwise." Furthermore, Downey claims "`std::optional` is filling too many roles" and is not "directly safe", unlike `std::maybe_view`.

While we support `views::nullable`, we find the addition of the `std::maybe_view` contrary to the principles of simplicity and genericity. Instead, we propose making `std::optional` a range.

This paper highlights the principles behind our design alternative, outlines the drawbacks of adding a new `std::optional`-like type, surveys existing practice, and demonstrates how our proposal simplifies all of P1255R12's examples.

Principles

Two design principles directly apply to the rangification of *`std::optional`*: genericity and simplicity.

Alex Stepanov, the progenitor of much of the C++ standard library, repeatedly emphasized the importance of genericity for software reuse. In *Fundamentals of Generic Programming* (<http://stepanovpapers.com/DeSt98.pdf>) he and James C. Dehnert wrote:

Generic programming recognizes that dramatic productivity improvements must come from reuse without modification, as with the successful libraries. Breadth of use, however, must come from the separation of underlying data types, data structures, and algorithms, allowing users to combine components of each sort from either the library or their own code. Accomplishing this requires more than just simple, abstract interfaces – it requires that a wide variety of components share the same interface so that they can be substituted for one another. It is vital that we go beyond the old library model of reusing identical interfaces with pre-determined types, to one which identifies the minimal requirements on interfaces and allows reuse by similar interfaces which meet those requirements but may differ quite widely otherwise. Sharing similar interfaces across a wide variety of components requires careful identification and abstraction of the patterns of use in many programs, as well as development of techniques for effectively mapping one interface to another.

Simplicity, the second applicable principle, makes C++ easier to learn and use, increasing its applicability to different problem domains. The direction group in *Direction for ISO C++ (P2000R4)* (<https://wg21.link/P2000R4>) sees "C++ in danger of losing coherency due to proposals

based on differing and sometimes mutually contradictory design philosophies and differing stylistic tastes." and David Sankel highlights the impact on training costs in *C++ Should Be C++* (<https://wg21.link/P3023R1>).

With genericity and simplicity in mind, let us now investigate the proposed `std::maybe_view<T>`. In essence, it "may or may not store a value of type `T` in its storage space". The wording used here is notably *identical* to that of the proposal that introduced `optional`^[1]. This duplication alone is cause for concern due to the complexity of having multiple standardized types representing the same thing.

Indeed, the interfaces of the two types have a substantial amount in common. In the table below, let `V` be some type, `Opt` be either `optional<V>` or `maybe_view<V>`, `o` and `o2` be objects of type `Opt`, and `v` be an object of type `V`:

<code>std::optional<V></code>	both	Proposed <code>std::maybe_view<V></code>
<code>Opt(nullopt)</code>	<code>Opt()</code>	
	<code>Opt(v)</code>	
	<code>Opt(in_place, v)</code>	
<code>o = nullopt;</code>	<code>o = v;</code>	
<code>o.emplace(v);</code>		
<code>o.reset();</code>		
<code>o.swap(o2);</code>		
	<code>o == o2</code> and <code>o <=> o2</code>	
	<code>o == v</code> and <code>o <=> v</code>	
<code>*o</code> and <code>o->m</code>	<code>o.transform(f)</code>	<code>o.begin()</code> and <code>o.end()</code>
<code>o.has_value()</code> and <code>bool(o)</code>	<code>o.and_then(f)</code>	<code>o.size()</code>
<code>o.value_or(v)</code>	<code>o.or_else(f)</code>	<code>o.data()</code>
<code>std::hash<Opt>{}(o)</code>		

Nevertheless, Downey argues that the interface differences make this new type worthwhile. Consider each of these differences:

1. **`std::maybe_view` lacks a dereference operator and boolean conversion.** Dereference and boolean conversion are the standard interface to optional-like objects and originate in nullable C pointers. Generic algorithms that today output both a `vector<optional<T>>` and a `vector<shared_ptr<T>>` using optional-like interfaces, will not work with a `vector<maybe_view<T>>`. This interface difference goes against genericity.

2. **`std::maybe_view` contains some, but not all, accessors.** `std::maybe_view` provides the continuation functions `transform`, `and_then`, and `or_else` but not `value_or`. What's the motivation for this omission? A GitHub search^[2] results in 111k uses of `value_or`. If we add future accessors to `std::optional`, what will be the basis for deciding whether they should be added to `std::maybe_view`?
3. **`std::maybe_view` 's template parameter supports references.** This is fixing a defect of `std::optional` that is already being fixed with Steve Downey and Peter Sommerlad's `std::optional<T&>` (<https://wg21.link/P2988R3>) paper.
4. **`std::maybe_view` satisfies the range concept.** This interface difference allows `std::maybe_view` to be used in more algorithms than `std::optional`, which is great. However, if this is the only viable interface difference, we should add range support to `std::optional` to avoid confusing users with two otherwise identical types. This interface difference goes against simplicity.

The standard providing two almost identical types will result in choice overload (<https://en.wikipedia.org/wiki/Overchoice>), which is mentally draining. Downey suggests using `std::maybe_view` when "the value will have operations applied if present, and ignored otherwise." For a return type, how could one know if this will be the case for a caller? For an argument type, this may be true for one implementation, but what if it changes? Users should not have to spend mental energy considering questions like these.

Survey of existing practice

Since its standardization, `std::optional` 's usage has increased in modern C++ codebases. Iterating over this standard type is a recurring topic as shown in forum discussions such as Iterating over `std::optional` (<https://stackoverflow.com/questions/72812599/iterating-over-stdoptional>) and begin and end iterators for `std::optional`? (https://www.reddit.com/r/cpp/comments/17tyd44/begin_and_end_iterators_for_stdoptional_how_to/).

Some in the C++ community have already experienced this idea. `owi_optional` (https://github.com/seleznevae/owi_optional) is an open-source implementation. The following code snippet shows an example using syntax similar to our proposal:

```
#include "owi/optional.hpp"

owi::optional<int> empty;
for (int i : empty) { // iterating over empty optional
    std::cout << i;
}

owi::optional<int> opt{ 123 };
for (int i : opt) { // iterating over non-empty optional
    std::cout << i;
}
```

Several other programming languages with `std::optional` -like types allow direct iteration. Rust's Option (<https://doc.rust-lang.org/std/option/#iterating-over-option>) type and Scala's Option (<https://www.scala-lang.org/api/2.13.5/scala/Option.htm>) are notable examples. The following code illustrates a typical Rust usage example:

```
let yep = Some(42);
let nope = None;
// chain() already calls into_iter(), so we don't have to do so
let nums: Vec<i32> = (0..4).chain(yep).chain(4..8).collect();
assert_eq!(nums, [0, 1, 2, 3, 42, 4, 5, 6, 7]);
let nums: Vec<i32> = (0..4).chain(nope).chain(4..8).collect();
assert_eq!(nums, [0, 1, 2, 3, 4, 5, 6, 7]);
```

Proposal

We propose a simple design alternative to P1255R12: make `std::optional` satisfy the `ranges::range` concept where iterating over a `std::optional` object will iterate over its 0 or 1 elements.

To view or not to view

As with the proposed `std::maybe_view`, `std::optional` should be a `view`. As it only ever has at most one element, it would satisfy the semantic requirements.

The question is *how* to opt `std::optional` into being a `view`. There are two ways to do this:

1. Change `std::optional<T>` to inherit from `std::ranges::view_interface<std::optional<T>>`, or
2. Specialize `std::ranges::enable_view<std::optional<T>>` to `true`.

The problem with inheriting from `ranges::view_interface`, separate from any questions of ABI and the like, are that it would bring in more functions into `std::optional`'s public interface: `empty()`, `data()`, `size()`, `front()`, `back()`, and `operator[]()`. While `empty()` is potentially sensible (if unnecessary given that both `has_value()` and `operator bool` exist), the rest are actively undesirable. Note that all the accessor objects in `std::ranges` will still work (e.g. `std::ranges::size`, `std::ranges::data`, etc.) simply from providing `begin()` and `end()`.

So we propose instead to specialize `ranges::enable_view` (similar to `std::string_view` and `std::span`).

`std::optional<T>` is not a borrowed range, although `std::optional<T&>` (when added) would be.

Iterator Choice

`optional<T>` is a contiguous, sized range, so the simplest choice of iterator is `T*`. There are some benefits to having distinct iterator types – notably `std::span<T>::iterator` could be `T*` but is specified to just be some contiguous iterator (and none of the three major standard library implementations use `T*`). But there, `std::span`'s primary purpose of existence is to be a range while here we're just providing functionality, so `T*` is good enough.

Wording

Add the specialization of `ranges::enable_view` to [optional.syn]:

```
namespace std {
    // [optional.optional], class template optional
    template<class T>
        class optional;

    + template<class T>
    +     constexpr bool ranges::enable_view<optional<T>> = true;

    // ...
}
```

Add the range accessors to [optional.optional.general], right before the observers:

```
namespace std {
    template<class T>
    class optional {
    public:
        using value_type = T;

        // ...

        // [optional.swap], swap
        constexpr void swap(optional&) noexcept(see below);

    + // [optional.iterators], iterator support
    + constexpr T* begin() noexcept;
    + constexpr const T* begin() const noexcept;
    + constexpr T* end() noexcept;
    + constexpr const T* end() const noexcept;
    +
    + constexpr reverse_iterator<T*> rbegin() noexcept { return reverse_iterator(end()); }
    + constexpr reverse_iterator<const T*> rbegin() const noexcept { return reverse_iterator(end()); }
    + constexpr reverse_iterator<T*> rend() noexcept { return reverse_iterator(begin()); }
    + constexpr reverse_iterator<const T*> rend() const noexcept { return reverse_iterator(begin()); }
    +
    + constexpr const T* cbegin() const noexcept { return begin(); }
    + constexpr const T* cend() const noexcept { return end(); }
    + constexpr reverse_iterator<const T*> crbegin() const noexcept { return rbegin(); }
    + constexpr reverse_iterator<const T*> crend() const noexcept { return rend(); }

        // [optional.observe], observers
        constexpr const T* operator->() const noexcept;
        // ...
    };
}
```

Add a new clause [optional.iterators]. Note that this wording is based on adopting the currently proposed resolution for LWG 4015 (<https://cplusplus.github.io/LWG/issue4015>):

```
constexpr T* begin() noexcept;
constexpr const T* begin() const noexcept;
```

Returns: If *has_val*, `addressof(val)` . Otherwise, unspecified.

```
constexpr T* end() noexcept;
constexpr const T* end() const noexcept;
```

Returns: `begin() + has_val` .

Appendix: Comparison P1255R12 vs P3168R0

This section compares examples from P1255R12 to equivalents using our own proposal.

Optional access

P1255R12	This Proposal
<pre>for (auto&& opt : views::nullable(get_optional())) { // a few dozen lines... use(opt); // opt is Safe }</pre>	<pre>for (auto&& opt : get_optional()) { // a few dozen lines... use(opt); // opt is Safe }</pre>

Optional assignment

P1255R12	This Proposal
<pre>auto opt_int = get_optional(); for (auto&& i : views::nullable(std::ref(opt_int))) { // a few dozen lines... i = 123; }</pre>	<pre>auto opt_int = get_optional(); for (auto&& i : opt_int) { // a few dozen lines... i = 123; }</pre>

Range chain example (1)

```
vector<int> v{2, 3, 4, 5, 6, 7, 8, 9, 1};
auto test = [](int i) -> optional<int> {
    switch(i) {
        case 1:
        case 3:
        case 7:
        case 9:
            return i;
        default:
            return {};
    }
};
```

P1255R12

```
auto&& r = v
| views::transform(test)
| views::transform(views::nullable)
| views::join |
| views::transform(
    [](int i) { cout << i; return i; }
);
```

This Proposal

```
auto&& r = v
| views::transform(test)
| views::filter([](auto x) { return bool(x); })
| views::transform([](auto x){ return *x; })
| views::transform(
    [](int i) { cout << i; return i; }
);
```

```
for (auto&& i : r) {
    std::cout << i << "\n";
}
```

Range chain example (2)

```
unordered_set<int> set{1, 3, 7, 9};
auto flt = [=](int i) -> optional<int> {
    if (set.contains(i))
        return i;
    else
        return {};
};
```

P1255R12

```
for(auto i : views::iota{1,10} | views::transform(flt)) {
    for (auto j : views::nullable(i)) {
        for (auto k : views::iota(0, j))
            cout << '\a';
        cout << '\n';
    }
}
```

This Proposal

```
for(auto i : views::iota{1,10} | views::transform(flt)
    for (auto j : i) { // no need to transform
        for (auto k : views::iota(0, j))
            cout << '\a';
        cout << '\n';
    }
}
```

yield_if

Eric Niebler's Pythagorean Triples (<https://ericniebler.com/2018/12/05/standard-ranges/>) triple example:

P1255R12	This Proposal
<pre>auto yield_if = []<class T>(bool b, T x) { return b ? maybe_view<T>{move(x)} : maybe_view<T>{}; };</pre>	<pre>auto yield_if = []<class T>(bool b, T x) { return b ? optional<T>{move(x)} : nullopt; };</pre>

References

- Fernando Cacciola and Andrzej Krzemiński. N3793: A proposal to add a utility class to represent optional objects (Revision 5). <https://wg21.link/N3793> (<https://wg21.link/N3793>), 10/2013.
- James C. Dehnert and Alexander Stepanov. Fundamentals of Generic Programming. Lecture Notes in Computer Science volume 1766. <http://stepanovpapers.com/DeSt98.pdf> (<http://stepanovpapers.com/DeSt98.pdf>), 1998.
- Steve Downey. P1255R12: A view of 0 or 1 elements: views::maybe. <https://wg21.link/p1255r12> (<https://wg21.link/p1255r12>), 01/2024.
- Steve Downey and Peter Sommerlad. P2988R3: std::optional<T>. <https://wg21.link/P2988R3>, 02/2024.
- Hinnant, Howard et al. "Direction for ISO C++." 15 October 2022, <https://wg21.link/P2000R4> (<https://wg21.link/P2000R4>).
- Eric Niebler et al. P0896R4: The One Ranges Proposal. <https://wg21.link/P0896R4> (<https://wg21.link/P0896R4>), 11/2018.
- David Sankel. P3023R1: C++ Should Be C++. <https://wg21.link/P3023R1> (<https://wg21.link/P3023R1>), 10/2023.
- Wikipedia Foundation. Overchoice. <https://en.wikipedia.org/wiki/Overchoice> (<https://en.wikipedia.org/wiki/Overchoice>), 27 February 2024.

1. See the introduction of *A proposal to add a utility class to represent optional objects*

(<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3793.html>) 

2. See GitHub search of '"value_or" path:*.cpp' (https://github.com/search?q=%22value_or%22+path%3A*.cpp&type=code) 

(https://github.com/search?q=%22value_or%22+path%3A*.cpp&type=code) 